1. Header, conventions, and D2 overview

Project title: Ad-Vocates

Team Members: Dylan Shackleford, Max Marsh

Goal: Create a reliable and efficient tool that improves browsing performance while giving users greater privacy, security, and control over their web experience.

Conventions: Boxes show actions in Figure D2-1. Arrows show processing order. Red arrows show failure paths. PK = primary key.

Overview: Ad-Vocates is a browser extension that blocks advertising and tracking requests, allowing users to view blocking statistics, allow certain websites, and customize protection settings. Filtering/blocking rules update from approved sources.

D1 covered the main data model, filtering algorithm, and core methods, while D2 finishes the remaining uncovered components and adds error handling ,requirements traceability, design rationale, and peer review notes.

Known Gaps: The production filter source and license still need approval. Browser/API and filter compatibility are open choices still from D1. General pop-up handling (outside of known malicious hosts) is an open scope decision. The D2 Filter Validator accepts hostname-per-line lists, so the chosen source must be in hostname-format or the validator needs to be expanded/altered to support other adblock lists.

2. Finished components, the D2 diagrams

D2 keeps the D1 entities and local storage methodology. The following are the four components left for D2, D1 entity additions used by D2 are listed after:

Component	**Data model**
Dashboard	No persistent data. Temp settings, stats, loading status, and error msgs
Statistic Recorder	D1 StatisticsSummary:statsId (string, PK), blockedTotal and estimatedBytes(integer). Add unknownEstimateTotal (integer) for

	events with no estimate and incomplete (boolean) for lost data.
Filter Updater	UpdateStatus: sourceId(string,PK), lastSuccessAt and nextAttemptAt (timestamps), failureCount (integer), lastErrorCode(string).
Filter Validator	No persistent data. Holds downloaded text/candidate FilterRule records during validation.
D1 additions (Settings, Logging, Configuration)	ApplicationSettings: add revision(integer) which is incremented on save and checked against expectedRevision. ErrorLog: add component (string) and errorCode(string) - errorMsg holds fixed msg for code. ApprovedSource: sourceId(string, PK) AND url (HTTPS string) is read by the Filter Updater and Filter Validator

Dashboard - Max Marsh
Data - No persistent data

1. Read settings, statistics and update status when opened. Show unavailable message and retry button if query fails/exceeds 5 seconds.
2. Send settings changes with displayed revision. Show success after saving, keeping previous values if save fails. After a timeout, re-read settings before submitting again.
3. Refresh statistics every 5 seconds while open. Label savings as estimated and mark incomplete totals.

Statistic Recorder - Dylan Shackleford

Data: StatisticsSummary; up to 1,000 pending events and 10,000 recent event IDs

1. Accept a blocking event, validate fields, and ignore recently processed IDs.
2. Increase blockedTotal by one. Add event's estimatedBytes to total, or increase unknownEstimateTotal if null.
3. Save pending totals after 100 events/5 seconds. Remove pending data after a successful transaction, or retry after 5 seconds if failure occurs.

Filter Updater - Dylan Shackleford

Data: UpdateStatus and an approved sourceId to HTTPS-URL config

1. Run once per day or when manually refreshed. Allow one update at a time and save the next update time. After restart, run one overdue update.
2. Download from approved URL with 30 second timeout and 10 MB size limit. Reject unapproved redirects and pass downloaded text to the Filter Validator.
3. Save success after new rules are active. If there is a network or save error, retry after 1, 4 and 24 hours.

Filter Validator - Dylan Shackleford

Data: No persistent data

1. Check approved source and accept UTF-8 text that contains one DNS hostname per line. Ignore blank lines and comments and normalize case.
2. Remove duplicates, reject invalid hostnames, empty data and invalid syntax.
3. Pass a complete, valid set of filters to Settings. A hostname rule matches the hostname and subdomains. Reject the whole set if any validation fails.

Filter Request - Max Marsh

Data: no added persistent data; uses active settings, allowlist, hostname rule index in memory

1. Read current config. Allow request if protection is disabled or top-level site can’t be identified.
2. Check the exact page hostname against allowlist. If allowlisted, allow request, Else, check the request hostname and parent domains against the rule index.
3. Return BLOCK and matchedRuleId for matches, else return ALLOW. Calculate estimatedBytes with measured avg for resourceType and send an I3 event, or null if no estimate is available.

Settings - Max Marsh

Data: ApplicationSettings, AllowlistEntry, FilterRule

1. Validate settings and normalize hostnames. Reject invalid or >10,000 unique entries.
2. Serialize changes and check expectedRevision. Reject outdated saves with REVISION_CONFLICT, and build replacement config prior to save.

3. Save related records in a single transaction, then replace in-memory active reference. Return success afterwards.

Local Storage - Max Marsh

Data: D1 entities and D2 additions

1. Open schema version 1 and restore saved records.
2. Validate reads and writes. Commit or abort related changes together.
3. Prune expired logs.

Supporting Functions

Function	**Data, interface and behavior**
Configuration	Handled in Settings through getSettings/updateSettings and approved source config. Invalid changes keep prior config.
Logging	ErrorLog stores fixed msgs, appendError({component,errorCode}) keeps at most 100 records and removes records older than 1 week. LoggingErrorHandler drops failed logs
Notifications	No persistent data. Dashboard showStatus({code,severity}) shows inline text.
Access checks	No persistent data. authorizeMessage(sender, method) accepts only Dashboard pages for public methods, MessageAccessHandler rejects other callers.

Interfaces and D0 Review

Messages use {version, requestId, method, payload}. Replies return {ok,data}, or {ok: false, error: {code, message,recoverable}}. RequestInput keeps D1 requestUrl, pageHost, resourceType, and requestId. SettingsChange contains enabled, optional allowlist and expectedRevision. BlockEvent contains eventId, category, and estimatedBytes. Settings includes enabled, allowlist, revision and protection status. Statistics include all summary fields. V1 is the initial contract, and D2 refines planned D1 contracts prior to implementation.

D0 Interface	**D2 Contract / errors**
I1 Browser to Filter Request	evaluateRequest(RequestInput) = decision, matchedRuleId, errorCode. INVALID_URL/FILTERS_UNAVAILABLE/ INTERNAL_ERROR allow the request to go through.
I2 Filter Request to Local Storage	loadSnapshot() during init; appendError(error). STORAGE_ERROR
I3 Filter Request to Statistic Recorder	recordBlockedEvent(BlockEvent) = accepted | duplicate | rejected. INVALID_EVENT rejects that specific event. QUEUE_FULL marks totals as incomplete
I4 Statistic Recorder to Local Storage	readStatistics() and applyStatisticsDelta(delta). Separate read/write calls. STORAGE_ERROR
I5 Dashboard to Statistic Recorder	getStatistics() = totals and incomplete status. If there are no usable totals, return STORAGE_ERROR
I6 Dashboard to Settings	getSettings(); updateSettings(SettingsChange). INVALID_SETTING, INVALID_HOSTNAME, LIMIT_EXCEEDED, REVISION_CONFLICT, STORAGE_ERROR
I7 Settings to Local Storage	loadConfiguration(); commitConfiguration(config) = revision. STORAGE_ERROR, QUOTA_EXCEEDED
I8 Filter Updater to Approved Source	fetchApprovedSource(sourceId) = filterText. NETWORK_ERROR, TIMEOUT, UNAPPROVED_SOURCE, DOWNLOAD_TOO_LARGE
I9 Filter Updater to Filter Validator	validateFilters(sourceId, filterText) = candidate. INVALID_FILTER_DATA, UNSUPPORTED_RULE, LIMIT_EXCEEDED

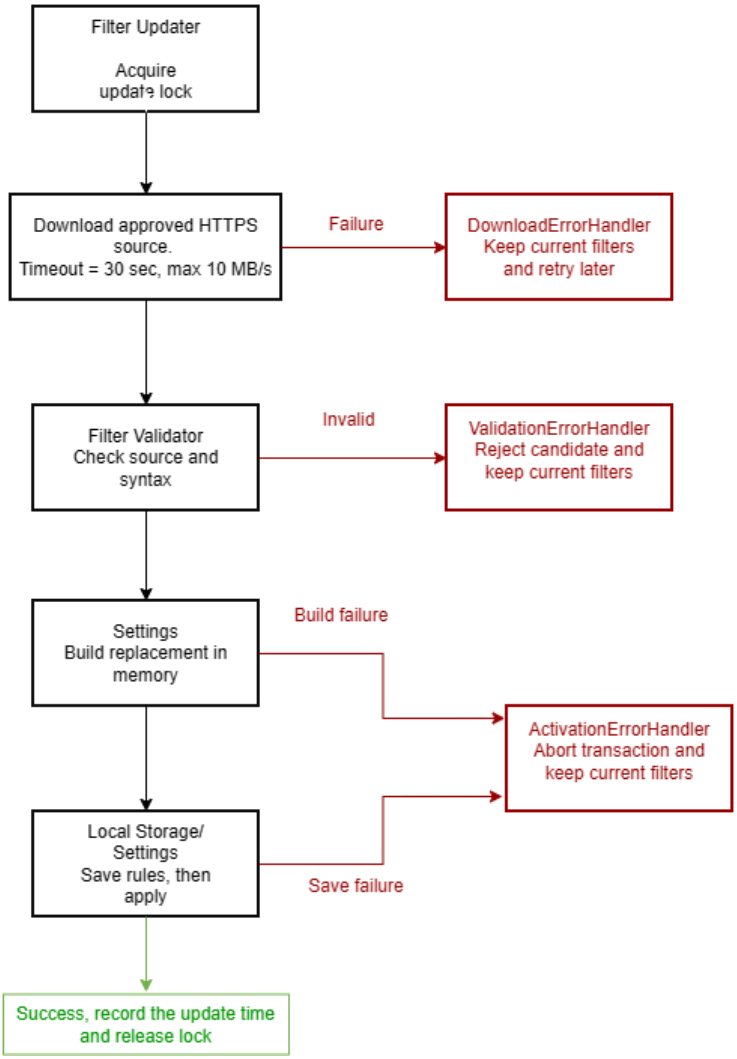
I10 Filter Validator to Settings	installValidatedFilters(candidate) = success, ruleCount, updatedAt. Error leaves current filters active.
I11 Settings to Filter Request	publishSnapshot(snapshot) replaces memory after commit
(New) I12 Dashboard to Filter Updater	getUpdateStatus(); refreshFilters(). UPDATE_IN_PROGRESS to stop overlaps.

3. Edge cases and error handling

Ad-Vocates

Figure D2-1. Filter Update and Error Paths

Goal: Create a reliable and efficient tool that improves browsing performance while giving users greater privacy, security, and control over their web experience.



ID	Component	Category	Edge Case	Design Decision
E01	Dashboard	Boundary	No statistics totals	DashboardData Handler shows zero if empty

				data has arrived, unavailable otherwise. Recoverable: yes when data arrives
E02	Dashboard	Timing	Out-of-order response	DashboardResponseHandler ignores older responses. Recoverable: yes, refresh
E03	Dashboard	Bad input	Invalid input	DashboardInputHandler shows error and retains old values. Recoverable: yes, correct input and resubmit
E04	Dashboard	External	Failure/timeout of messaging	DashboardErrorHandler shows retry, reread timed-out save. Recoverable: yes, when messaging is online
E05	Settings	Boundary	Empty or duplicate or too large allowlist	SettingsValidationHandler accepts empty, removes dupes, rejects >10,000 entries. Recoverable: yes
E06	Settings	Timing	Saves overlap	SettingsWriteHandler applies one change at a time, rejects stale revisions. Recoverable: yes. reread

E07	Settings	Bad input	Invalid hostname	SettingsValidationHandler rejects and doesn't save. Recoverable: yes, corrected input
E08	Settings	External	Save fails	ActivationErrorHandler keeps previous config and shows error. Recoverable: yes, retry
E09	Filter Request	Boundary	No rules	RequestBoundaryHandler allows requests with no filters. Recoverable: yes, load rules
E10	Filter Request	Timing	Config changes	RequestLifecycleHandler uses one config per request. Recoverable: yes, original config completes request
E11	Filter Request	Bad input	Invalid URL	RequestErrorHandler allows. Recoverable: degraded operation, request bypasses the filtering
E12	Filter Request	External	Missing permissions	BrowserAdapterHandler shows unavailable. Recoverable: if permissions change
E13	Statistic	Boundary	Counter/event	StatsLimitHandl

	Recorder		overflows	er caps totals and drops excess events. Recoverable: degraded operation, data is lost
E14	Statistic Recorder	Timing	Duplicate event	StatsEventHandler deduplicates recent IDs. Recoverable: yes, duplicates ignored
E15	Statistic Recorder	Bad input	Missing ID, negative value	StatsValidationHandler rejects. Recoverable: degraded operation, data lost
E16	Statistic Recorder	External	Storage fails	StatsErrorHandler holds pending data and retries after 5 sec. Recoverable: while data is in memory, yes - otherwise, data lost.
E17	Local Storage	Boundary	Retention limit hit	StorageLimitHandler prunes old logs, retries once. Recoverable: yes, old logs removed
E18	Local Storage	Timing	Concurrent writes	StorageTransactionHandler serializes writes. Recoverable: yes
E19	Local Storage	Bad input	Corrupt record/invalid	StorageSchemaHandler

			schema	preserves records and shows error. Recoverable: no
E20	Local Storage	External	Database unable to open	StorageErrorHandler keeps cached data and retries. Recoverable: yes, when database opens
E21	Filter Updater	Boundary	Empty or too large download	DownloadLimitHandler rejects either scenario. Recoverable: no, valid download needed
E22	Filter Updater	Timing	Overlapping update	UpdateLockHandler runs one update at a time. Recoverable: yes
E23	Filter Updater	Bad input	Unknown source	UpdateInputHandler rejects. Recoverable: no, need correct source
E24	Filter Updater	External	Timeout	DownloadErrorHandler retries 1/4/24hr intervals. Recoverable: yes if retry is successful
E25	Filter Validator	Boundary	Empty or dupe rules	ValidationLimitHandler dedupes and rejects empty rules. Recoverable: duplicates yes, empty no

E26	Filter Validator	Timing	Repeated candidate	ValidationLifecycleHandler ignores duplicate/repeated candidates. Recoverable: yes
E27	Filter Validator	Bad input	Invalid syntax	ValidationErrorHandler rejects complete candidate. Recoverable: no, need correct syntax candidate
E28	Filter Validator	External	Source sends unexpected content	SourceValidationHandler checks content and validates, keeping old rules if invalid. Recoverable: no, need corrected source.
E29	Logging	Boundary	Record cap reached	LoggingErrorHandler removes oldest record before appending. Recoverable: yes, oldest log is lost
E30	Logging	Timing	Multiple components log at same time	StorageTransactionHandler serializes writes. Recoverable: yes
E31	Logging	Bad input	Unknown errorCode	LoggingErrorHandler stores INTERNAL_ERROR. Recoverable: yes
E32	Logging	External	Write fails	LoggingErrorHandler drops log,

				caller continues. Recoverable: degraded operation, data lost
E33	Access checks	Boundary	Unsupported message version	MessageAccess Handler rejects, returns error. Recoverable: yes, after extension refresh
E34	Access checks	Timing	Message arrives before components load	MessageAccess Handler returns error, Dashboard retries. Recoverable: yes
E35	Access checks	Bad input	Unknown method	MessageAccess Handler rejects, no state change. Recoverable: no, caller not permitted
E36	Access checks	External	Sender info unavailable	MessageAccess Handler rejects message. Recoverable: yes, when messaging returns
E37	Notifications	Boundary	Multiple statuses at same time	Dashboard displays most recent highest severity status. Recoverable: yes
E38	Notifications	Timing	Stale error after success	Dashboard clears status on successful reply. Recoverable: yes
E39	Notifications	Bad input	Unknown status	Dashboard shows generic

				error. Recoverable: yes
E40	Notifications	External	Dashboard closed when error happens	Error remains in ErrorLog, status re-read when Dashboard opens. Recoverable: yes

Configuration edge cases are handled through E05-E08 and E23.

4. Requirements traceability table

ID and Req.	**Priority**	**Design element**	**How**
US-01: Block ads and trackers	High	evaluateRequest; I1/I3; E09-E11, E14	Block matched requests, count each one time
US-02: Show blocking statistics	High	StatisticsSummary; getStatistics; E01, E13-E16	Display blocked totals and estimated data savings.
US-03: Allow trusted sites	High	AllowlistEntry; isAllowlisted; E05, E07	Bypass filtering for saved hostnames
US-04: Update filter rules	High	refreshFilters; validateFilters; Fig D2-1; E21-E28	Apply valid updates and retain old rules on failure
US-05: Recover broken site functions	High	updateSettings; AllowlistEntry; E05-E08	Allow sites without disabling protection for others
FR-01: Suspicious requests and redirects	High	evaluateRequest; E10, E11	Block requests that match rules
FR-02: Popup handling	Deferred	N/A	Need to decide how to handle general popups - malicious hosts blocked, but general rule needed
FR-03: Protection	High	ApplicationSettings	Save protection

controls		enabled; updateSettings; E06,E08	toggles separately from site exceptions
NFR-01: Minimal data collection	High	StatisticsSummary; ErrorLog; E17, E31	Keep local totals and exclude URLs from logs
NFR-02: Verified updates	High	ApprovedSource; validateFilters; E23, E27, E28	Check sources and syntax prior to applying updates
NFR-03: Clear settings and estimates	High	Dashboard status text; StatisticsSummary; E01, E13, E37-E39	Label settings clearly and savings as estimates
NFR-04: Usability during failures	High	RequestErrorHandler; Fig D2-1; E08,E11,E24, E28	Allow on filtering failure, retain current rules if update fails
NFR-05: Low resource use	Medium	In-memory rule index; bounded queues; E13, E16	Avoid storage calls during filtering process, limit pending data amounts.

Functional and non-functional requirements were formalized from the project's scope into labeled requirements for this table.

Reverse check

Component	**Requirement**
Dashboard	US-02, US-03, US-05, FR-03, NFR-03
Settings	US-03, US-04, US-05, FR-03
Filter Request	US-01, US-03, FR-01, NFR-04, NFR-05
Statistic Recorder	US-01, US-02, NFR-03, NFR-05
Local Storage	US-02, US-03, US-04, US-05, NFR-01
Filter Updater	US-04, NFR-02, NFR-04
Filter Validator	US-04, NFR-02

Configuration	FR-03, NFR-02
Logging	NFR-01, NFR-04
Notifications	NFR-03, NFR-04
Access checks	US-03, US-05, FR-03

All components trace to at least one requirement; no scope creep.

5. Design rationale

The D0 decision log (separation of Filter updater and Validator, local filtering, etc) is referenced instead of repeated. R3 builds on local filtering and R5 on totals-only stats.

R1. We chose a set for allowlist lookup over an array because lookups need to be fast. We accepted the trade-off of extra memory due to the frequency of checking hostnames.

R2. We chose complete config saves over partial changes because settings and rules need to be consistent. We accepted the trade-off of more thorough save handling because a failed update must preserve the current working values.

R3. We chose loaded rules over storage reads per request because filtering should not be affected by storage delays. We accepted the trade-off of memory use since rules are reused across requests.

R4. We chose allowing requests on a failure over blocking them because websites should remain usable if the blocker fails. We accepted the trade-off of reduced protection because a failure of our blocker should not prevent browsing.

R5. We chose buffered statistics over saving every event immediately because storage should not delay filtering. We accepted the trade-off of possible unsaved counts/data-loss since statistics are diagnostic and not essential.

R6. We chose labeled estimates over claims of exact savings because blocked content is not downloaded. We accepted the trade-off of limited precision because the displayed data must be honest to the user and estimates allow for greater user privacy through minimal data collection.